# CST1510 Programming for data communication and networks

## Week 4: Strings, Special Methods, Modules, Try-Except statment and (optional) Regular Expressions

## WORKSHOP

---

# STRINGs

## String in Python

It is one of the primitive data structures and are the building blocks for data manipulation. Python has a built-in string class named str Python strings are "immutable" which means they **cannot be changed after they are created.**

---

### Immutable string

A string object is immutable. Its contents cannot be changed, its value cannot be updated, only the reference to the string

No description has been provided for this image

---

In [ ]:
```
s1 = 'Welcome'
print(id(s1))
```

In [ ]:
```
s2 = 'Welcome'
print(id(s2))
```

In [ ]:
```
s1[0] = 'w' #  TypeError: 'str' object does not support item assignment
```

In [ ]:
```
# Examle of Mutable-objects- which could be freely modified

# list
l = [1,2,3]

# dictionary
d = {1:'one', 2:'two'}
```

```
# set
s = {1,2,3}
```

In [ ]:
```
# Examle of  Immutable objects which could not be freely modified

# string
s = '123'

# tuple
t = (1,2,3)

# frozen set
fs = frozenset([1,2,3])
```

## Creating string

In [ ]:
```
s1 = str() # Create an empty string object
```

In [ ]:
```
s2 = str("Welcome") # Create a string object for Welcome
```

In [ ]:
```
s1 = ' ' # Same as s1 = str()
```

In [ ]:
```
s2 = 'Welcome' # Same as s2 = str("Welcome")
```

Computers use binary numbers internally. A character is stored in a computer as a sequence of 0s and 1s. Mapping a character to its binary representation is called character encoding. There are different ways to encode a character. How characters are encoded is defined by an encoding scheme. One popular standard is **ASCII (American Standard Code for Information Interchange)**, a 7-bit encoding scheme for representing all uppercase and lowercase letters, digits, punctuation marks, and control characters. ASCII uses numbers 0 through 127 to represent characters.

The items of a string are **characters**. There is no separate character type; a string of one item represents a character. Characters represent (at least) 8-bit bytes. The built-in functions **chr()** and **ord()** convert between characters and nonnegative integers representing the byte values.

Bytes with the values 0-127 usually represent the corresponding ASCII values, but the interpretation of values is up to the program. The string data type is also used to define arrays of bytes, e.g., to hold data read from a file.

No description has been provided for this image
ASCII characters can fit in 8 bits, but Unicode characters need a lot more room UTF-32, for example, uses 32 bits. Remember 8 bits in a byte.2 to the power 8 = 256 if you want to think about bit manipulation a bit. 2 to the power 7 = 128.

Bit manipulation is an efficient way to represent numbers (and ASCII characters) in production. 0000000 7 empty bits, 0100000 2 to the power of 6 =64 chr(64) #--> returns @ this is the @ sign.

---

The **chr()** convert from ASCII value to character

In [ ]:
```
chr(64)
```

The **ord()** conver from character to ASCII value

In [ ]:
```
ord('@')
```

---

## Activity 1

Encode the text = 'Hello' into ASCII

In [ ]:
```

```

In [ ]:
```

```

## Built-in Functions for String

Strings support many of the same sequence operations as lists

| Function | Description |
|---|---|
| `len()` | Retrieves the length of a string. |
| `str()` | Converts an object into a user-friendly string representation. |
| `repr()` | Converts an object into a developer-friendly string representation. |
| `format()` | Provides advanced string formatting capabilities. |
| `ord()` | Converts a character into its corresponding integer value. |
| `chr()` | Converts an integer into its corresponding character. |
| `min()` | Returns the smallest character (based on ASCII value) in a string. |
| `max()` | Returns the largest character (based on ASCII value) in a string. |

In [ ]:
```
s = "Welcome"
```

The **len()** function to return the number of the characters in a string.

In [ ]:
```
len(s) # Since s has 7 characters, len(s) returns 7
```

The **repr()** provides a string that can be used to recreate the object or gives more precise details about it.

In [ ]:
```
repr(s)
```

The **format()** allows you to insert variables and specify how they should be displayed within a string, making it easier to create dynamic and readable output.

In [ ]:
```
# Example 1: Basic string formatting
name = "Alice"
age = 25
print("My name is {} and I am {} years old.".format(name, age))
# Output: My name is Alice, and I am 25 years old.

# Example 2: Formatting numbers
price = 49.99
print("The price is {:.2f} dollars.".format(price))
# Output: The price is 49.99 dollars.
```

The **max()** and **min()** functionsreturn the largest or smallest character in a string.

In [ ]:
```
max(s) # Note that the lowercase letters have a higher ASCII value than t
```

In [ ]:
```
min(s) # min(s) returns W
```

## Getting element from a string

In [ ]:
```
# Direct Print: This prints the entire string.
print(s)
```

In [ ]:
```
# Using a for Loop: This prints each character in the string individually
for i in s:
    print(i)
```

In [ ]:
```
#Using range() and Indexing: This also prints each character but uses the
for i in range(len(s)):
    print(s[i])
```

In [ ]:
```
#Using range() and Indexing: This also prints each character but uses the
for i in range(len(s)):
    print(s[i], end='')
```

## The Concatenation (+) and Repetition (*) Operators

You can join, or concatenate, two strings by using the concatenation operator (+). You can also use the repetition operator (*) to concatenate the same string multiple times.

In [ ]:
```
s1 = 'Welcome'
s2 = 'Python'
print(s1 + s2)
```

In [ ]:
```
print(s1*3)
```

---

## Activity 2

Create function returning one string of binary, representing a giving word, but first..

- Encode the text = ‘Hello’ into ASCII

- Find binary value for each of the ASCII number using the bin() cast that will return binary string
- Remove the '0b' prefix from each of the string
- Create s string and concatenate each of the string into one string
  You can change the above four steps into body of the function, pass text as a parameter and return the s variable.

In [ ]:

---

## in ... not in

In [ ]:
```
s1 = 'Welcome'

print('come' in s1)
print('come on' in s1)
```

---

## Comparing Strings

You can compare strings by using the comparison operators (==,!=, >, >=, <, and <=, introduced. Python compares strings by comparing their corresponding characters, and it does this by evaluating the characters' numeric codes. For example, a is larger than A because the numeric code for a is larger than the numeric code for A. The ASCII Character Set, to find the numeric codes for characters.

In [ ]:
```
print('a' > 'A')
```

In [ ]:
```
print(ord('a') , ord('A'))
```

In [ ]:
```
print("green" == "glow")
```

In [ ]:
```
print("green" != "glow")
```

In [ ]:
```
print("green" > "glow")
```

In [ ]:
```
print("ab" <= "abc")
```

The **==** operator compares the value or equality of two objects, whereas the Python **is** operator checks whether two variables point to the same object in memory. In the vast majority of cases, this means you should use the equality operators == and !=, except when you're comparing to None.

## Splitting a String into a List

The **.split** method, which is useful for splitting items in a string into a list.

In [ ]:
```
items = "Jane John Peter Susan".split(" ") # we use space delimeter
items
```

In [ ]:
```
items = "09/20/2012".split("/") # You can use a nonspace delimiter
print(items)
```

---

## Activity 3

Consider the following string : 'Hello world'

Create a list contaning letters of alphabet

Consider key = 7

1. Find the plaintext letter's index number
2. Add the key to the plaintext letter's number.
3. If this number is larger than 26, subtract 26.
4. Find the letter for the number you've calculated.

This is the ciphertext letter.

---

In [ ]:
```

```

---

## Slicing Strings and Indexing

- Indexing: Retrieves a specific character from a string using its index.
- Slicing: Extracts a segment of a string based on specified index ranges.

In [ ]:
```
s = 'Welcome'
```

In [ ]:
```
# Indexing
s[0]
```

In [ ]:
```
s[1]
```

In [ ]:
```
s[-1]
```

In [ ]:
```
s[2]
```

The slicing operator returns a slice of the string using the syntax **s[start : end]** The slice is a substring from index start to index end – 1.

In [ ]:
```
s[:2]
```

In [ ]:
```
s[-2:]
```

In [ ]:
```
s[::-1]
```

## `str` Methods

- https://docs.python.org/3/library/stdtypes.html#str
- https://docs.python.org/3/library/string.html#template-strings

| Method | Description |
|---|---|
| `capitalize()` | Converts the first character of the string to uppercase. |
| `casefold()` | Transforms the string to lowercase in a way suitable for case-insensitive comparisons. |
| `center()` | Centers the string within a specified width. |
| `count()` | Counts how many times a specific value appears in the string. |
| `encode()` | Returns an encoded version of the string using the specified encoding. |
| `endswith()` | Checks if the string ends with the specified suffix. |
| `expandtabs()` | Sets the tab width for the string. |
| `find()` | Searches for a specified value and returns the index of its first occurrence. |
| `format()` | Inserts specified values into the string using placeholders. |
| `format_map()` | Similar to `format()`, but takes a dictionary-like object for substitutions. |
| `index()` | Similar to `find()`, but raises an error if the value is not found. |
| `isalnum()` | Returns True if the string contains only letters and digits. |
| `isalpha()` | Returns True if the string contains only alphabetic characters. |
| `isascii()` | Returns True if the string contains only ASCII characters. |
| `isdecimal()` | Returns True if the string contains only decimal characters. |
| `isdigit()` | Returns True if the string contains only numeric digits. |
| `isidentifier()` | Returns True if the string is a valid Python identifier. |
| `islower()` | Returns True if all characters in the string are lowercase. |
| `isnumeric()` | Returns True if the string contains only numeric characters. |
| `isprintable()` | Returns True if all characters in the string are printable. |
| `isspace()` | Returns True if the string contains only whitespace characters. |
| `istitle()` | Returns True if the string follows title case convention. |
| `isupper()` | Returns True if all characters in the string are uppercase. |
| `join()` | Concatenates elements of an iterable into a single string, separated by the specified delimiter. |
| `ljust()` | Left-aligns the string within a specified width. |

| Method | Description |
|---|---|
| `lower()` | Converts all characters in the string to lowercase. |
| `lstrip()` | Removes leading whitespace or specified characters from the string. |
| `maketrans()` | Creates a translation table for use with the `translate()` method. |
| `partition()` | Splits the string into three parts based on a separator: before, separator, and after. |
| `replace()` | Replaces occurrences of a specified value with another value. |
| `rfind()` | Searches for a specified value and returns the index of its last occurrence. |
| `rindex()` | Similar to `rfind()`, but raises an error if the value is not found. |
| `rjust()` | Right-aligns the string within a specified width. |
| `rpartition()` | Splits the string into three parts, but starts searching from the end. |
| `rsplit()` | Splits the string into a list, starting from the right and based on a separator. |
| `rstrip()` | Removes trailing whitespace or specified characters from the string. |
| `split()` | Splits the string into a list, using a specified separator. |
| `splitlines()` | Splits the string at line breaks and returns a list of lines. |
| `startswith()` | Checks if the string starts with the specified prefix. |
| `strip()` | Removes leading and trailing whitespace or specified characters from the string. |
| `swapcase()` | Swaps uppercase characters to lowercase and vice versa. |
| `title()` | Converts the first letter of each word to uppercase. |
| `translate()` | Returns a string where each character is mapped according to a translation table. |
| `upper()` | Converts all characters in the string to uppercase. |
| `zfill()` | Pads the string with zeros on the left, up to the specified length. |

**Note:** All string methods return a new string; they do not modify the original string.

- `capitalize()` Converts the first character of the string to uppercase.

In [ ]:
```
"hello".capitalize()
```

- `casefold()` Converts the string to lowercase for case-insensitive comparisons.

In [ ]:
```
"HELLO".casefold()
```

- `center()` Centers the string within a specified width. "hello".center(10) → ' hello '

In [ ]:
```
"hello".center(10)
```

- `count()` Counts how many times a specific value appears in the string.

In [ ]:
```
"hello world".count('o')
```

In [ ]:
```

```

- `encode()` Returns an encoded version of the string using the specified encoding.

In [ ]:
```
"hello".encode('utf-8')
```

- `endswith()` Checks if the string ends with the specified suffix.

In [ ]:
```
"hello".endswith('o')
```

- `expandtabs()` Sets the tab width for the string.

In [ ]:
```
"a\tb\tc".expandtabs(4)
```

- `find()` Searches for a specified value and returns the index of its first occurrence.

In [ ]:
```
"hello".find('l')
```

- `format_map()` Similar to format(), but takes a dictionary-like object for substitutions.

In [ ]:
```
"Name {name}".format_map({'name': 'John'})
```

- `index()` Similar to find(), but raises an error if the value is not found.

In [ ]:
```
"hello".index('l')
```

### Testing Strings

- The **isalnum()** returns True if characters in this string are either alphabet or number

In [ ]:
```
# string contains either alphabet or number
name1 = "Python3"
```

```
print(name1.isalnum()) #True
```

In [ ]:
```
# string contains whitespace
name2 = "Python 3"
print(name2.isalnum()) #False
```

- The **isalpha()** returns True if characters in this string are alphabetic and there is at least one character.

In [ ]:
```
name = "Monica"
print(name.isalpha())
```

In [ ]:
```
# contains whitespace
name = "Monica Geller"
print(name.isalpha())
```

In [ ]:
```
# contains number
name = "Mo3nicaGell22er"
print(name.isalpha())
```

- The **isdigit()** returns True if this string contains only number characters.

In [ ]:
```
str1 = '342'
print(str1.isdigit()) # True
```

In [ ]:
```
str2 = 'python'
print(str2.isdigit()) # False
```

In [ ]:
```

```

- The **isidentifier()** returns True if this string is a Python identifier.

In [ ]:
```
str = 'Python'
print(str.isidentifier())
```

In [ ]:
```
str = 'Py thon'
print(str.isidentifier())
```

In [ ]:
```
str = '22Python'
print(str.isidentifier())
```

In [ ]:
```
str = ''
print(str.isidentifier())
```

- The **islower()** returns True if all characters in this string are lowercase lettters and there is at least one character.

In [ ]:
```
s = 'this is good'
print(s.islower())
```

In [ ]:
```
s = 'th!s is a1so g00d'
print(s.islower())
```

In [ ]:
```
s = 'this is Not good'
print(s.islower())
```

In [ ]:
```
s = 'this is good'
if s.islower() == True:
  print('Does not contain uppercase letter.')
else:
  print('Contains uppercase letter.')
```

In [ ]:
```
s = 'this is Good'
if s.islower() == True:
  print('Does not contain uppercase letter.')
else:
  print('Contains uppercase letter.')
```

- The **isupper()** returns True if all characters in this string are uppercase lettters and there is at least one character.

In [ ]:
```
# example string
string_ = "THIS IS GOOD!"
print(string.isupper());
```

In [ ]:
```
# numbers in place of alphabets
string_ = "THIS IS ALSO G00D!"
print(string.isupper());
```

In [ ]:
```
# lowercase string
string_ = "THIS IS not GOOD!"
print(string.isupper());
```

In [ ]:
```
string_ = 'THIS IS GOOD'
if string_.isupper() == True:
  print('Does not contain lowercase letter.')
else:
  print('Contains lowercase letter.')
```

In [ ]:
```
string_ = 'THIS IS gOOD'
if string_.isupper() == True:
  print('Does not contain lowercase letter.')
else:
  print('Contains lowercase letter.')
```

- The **isspace()** returns True if this string contains only whitespace characters.

In [ ]:
```
s = ' '
print(s.isspace())
```

In [ ]:
```
s = ' a'
print(s.isspace())
```

In [ ]:
```
s = ''
print(s.isspace())
```

In [ ]:
```
s = '\t  \n'
if s.isspace() == True:
  print('All whitespace characters')
else:
  print('Contains non-whitespace characters')
```

In [ ]:
```
s = '2+2 = 4'
if s.isspace() == True:
  print('All whitespace characters')
else:
  print('Contains non-whitespace characters.')
```

## Activity 5

Consider the following password: 'Lei6i@RHt*9a3u' What can you do to check if the password contains the following:

the password is at least 8 characters long the password contains one upper case letter, the password contains one lower case letter, the password contains one digit the password contains one special character (example: '@')

---

In [ ]:

In [ ]:

---

## Searching for Substrings

The **endswith(s1: str):** returns True if the string ends with the substring s1.

In [ ]:
```
message = 'Python is fun'

# check if the message ends with fun
print(message.endswith('fun'))
```

In [ ]:
```
text = "Python is easy to learn."
result = text.endswith('to learn')
# returns False
print(result)
```

In [ ]:
```
result = text.endswith('to learn.')
# returns True
print(result)
```

In [ ]:
```
result = text.endswith('Python is easy to learn.')
# returns True
print(result)
```

In [ ]:
```
text = "Python programming is easy to learn."

# start parameter: 7
# "programming is easy to learn." string is searched
result = text.endswith('learn.', 7)
print(result)

# Both start and end is provided
# start: 7, end: 26
# "programming is easy" string is searched

result = text.endswith('is', 7, 26)
# Returns False
print(result)

result = text.endswith('easy', 7, 26)
# returns True
print(result)
```

---

The **startswith(s1: str):** returns True if the string starts with the substring s1.

In [ ]:
```
message = 'Python is fun'

# check if the message starts with Python
print(message.startswith('Python'))
```

In [ ]:
```
text = "Python is easy to learn."

result = text.startswith('is easy')
# returns False
print(result)

result = text.startswith('Python is ')
# returns True
print(result)

result = text.startswith('Python is easy to learn.')
# returns True
print(result)
```

In [ ]:
```
text = "Python programming is easy."

# start parameter: 7
# 'programming is easy.' string is searched
result = text.startswith('programming is', 7)
print(result)

# start: 7, end: 18
# 'programming' string is searched
result = text.startswith('programming is', 7, 18)
print(result)

result = text.startswith('program', 7, 18)
print(result)
```

---

The **find(s1)** returns the lowest index where s1 starts in this string, or –1 if s1 is not found in this string.

In [ ]:
```
message = 'Python is a fun programming language'

# check the index of 'fun'
print(message.find('fun'))
```

In [ ]:
```
quote = 'Let it be, let it be, let it be'

# ferst occurance of 'let it'(case sensitive)
result = quote.find('let it')
print("Substring 'let it':", result)

# find returns -1 if substring not found
result = quote.find('small')
print("Substring 'small ':", result)

# How to use find()
if (quote.find('be,') != -1):
    print("Contains substring 'be,'")
else:
    print("Doesn't contain substring")
```

### Raw Strings

- Backslash characters in strings introduce *escape sequences*—like `\n` for newline and `\t` for tab
- To include a backslash in a string, use two backslash characters `\\`
- Makes some strings difficult to read
- Consider a Microsoft Windows file location:

In [ ]:
```
file_path = 'C:\\MyFolder\\MySubFolder\\MyFile.txt'
```

In [ ]:
```
file_path
```

- **raw strings**—preceded by the character `r`—are more convenient
- They treat each backslash as a regular character, rather than the beginning of an escape sequence

In [ ]:
```
file_path = r'C:\MyFolder\MySubFolder\MyFile.txt'
```

In [ ]:
```
file_path
```

## `import`: A Deeper Look

### Importing Multiple Identifiers from a Module

- Use `from…import` to import a comma-separated list of identifiers from a module then use them without having to precede them with the module name and a dot ( `.` ):

In [ ]:
```
from math import ceil, floor
```

In [ ]:
```
ceil(10.3)
```

In [ ]:
```
floor(10.7)
```

### Caution: Avoid Wildcard Imports

- Import *all* identifiers defined in a module with a **wildcard** `import`.
- Makes all of the module's identifiers available.
- Can lead to subtle errors.
- Considered a dangerous practice.

In [ ]:
```

```

In [ ]:
```
from math import *
```

In [ ]:
```
e
```

After executing the import, variable `e` is replaced, possibly by accident, with the `math` module's constant `e`.

#### Binding Names for Modules and Module Identifiers

- Sometimes it's helpful to import a module and use an abbreviation for it to simplify your code.
- The `import` statement's `as` clause allows you to specify the name used to reference the module's identifiers.

In [ ]:
```
import statistics as stats
```

In [ ]:
```
data = [85, 93, 45, 87, 93]
```

In [ ]:
```
stats.mean(data)
```

In [ ]:
```

```

# Files and exception handling

### Errors and Exceptions :

- Handling exception
- Raising an exception

# Errors and Exceptions

"Errors detected during execution are called exceptions and are not unconditionally fatal"

What? Exceptions are raised when the program encounters an error during its execution. They disrupt the normal flow of the program and usually end it abruptly. To avoid this, you can catch them and handle them appropriately.

### Why handling exceptions? Why is this helpful?

By handling exceptions, you can provide an alternative flow of execution to avoid crashing your program unexpectedly.

There are (at least) two distinguishable kinds of errors: syntax errors and exceptions (doc https://docs.python.org/3/tutorial/errors.html)

**Syntax errors** are mistakes in the structure of Python code. They are caught during a special parsing stage before a program is executed.

**Exceptions** are runtime errors because they occur during program execution, only when the offending code (the code causing the error) is reached. It can occur with syntactically correct code.

In [ ]:
```
# Exmple of syntax error
print('Welcome to')
store = 'Hello World!'
print(stor))
```

There are many different types of exceptions, and they are all raised in particular situations Note that there is a lot of exceptions built into the language of Python. Again, we don't need to memorize all of them, but at some point, we may see them pop up in our programs. We can find details on each of the exceptions listed above in the https://docs.python.org/3/library/exceptions.html#built-in-exceptions

In [ ]:
```
# Example of situation when divide by zero
result = 5/0
```

In [ ]:
```
# Example of situation when trying to access element that is not there
wish = 'Wishing you good day!'
print(wish[500])
```

In [ ]:
```
# Example of situation when trying to access element that is not there
list_a = [1,2,3]
print(list_a[3])
```

In [ ]:
```
# To show the error message enter a character
number = int(input('Please enter a number'))
print(number)
```

## Handled exceptions using the try statement:

```
try:
    # Code that may raise an exception
except <ExceptionType>:
    # Code that runs if an exception occurs in the try block
else:
    # Code that runs if no exceptions were raised in the try
block
finally:
    # Code that always runs, no matter what (after try and except
blocks)
```

- The `try` block contains the code that you want to execute and potentially catch errors from.
- The `except` block handles specific exceptions that are raised in the try block. You can specify a particular type of exception (e.g., except ValueError:) or catch all exceptions.
- The `else` block runs only if no exception was raised in the try block.
- The `finally` block always executes, regardless of whether an exception was raised or not, often used for cleanup activities like closing files or releasing resources.

In [ ]:
```
# Example handling zero devision error
try:
    result = 5/0
except ZeroDivisionError as e:
    print(e)
```

In [ ]:
```
# Example of handling an error where data from a string try to access tha
wish = 'Wishing you good day!'
try:
    print(wish[500])
except IndexError as e:
    print(e)
```

In [ ]:
```
#Example of handling incorrect value type
while True:
    try:
        number = int(input('Please enter a number'))
    except ValueError as e:
        print(e)
    else:
        print(number)
    break
```

In [ ]:
```
#Example of handling incorrect value type with 'finally'
while True:
    try:
        number = int(input('Please enter a number'))
    except ValueError as e:
        print(e)
    else:
        print(number)
```

```
        break
    finally:
        print('Thank you !')
```

In [ ]:
```
# Additional Material: Raising Custom Exceptions with Strings
def validate_string(s):
    if not isinstance(s, str):
        raise TypeError("Only strings are allowed")
    if len(s) < 5:
        raise ValueError("String length must be at least 5 characters")

try:
    validate_string(12345)  # Testing with non-string input
except TypeError as e:
    print(e)

try:
    validate_string("abc")  # Testing with a short string
except ValueError as e:
    print(e)
```

## Raising an Exception

We can throw an custom exception at any time by using the raise keyword, even when Python would not normally throw it.

We might want to raise an exception anytime we think a mistake has or will occur in our program. This lets us stop program execution immediately and provide a useful error message instead of allowing mistakes to occur that may be difficult to diagnose at a later point.

```
# ... condition under which we will raise the exception
raise Exception('Error message')
```

In [ ]:
```
# Example of raising custom error
def open_register(employee_status):
    if employee_status == 'Authorized':
        print('Successfully opened cash register')
    else:
    # Alternatives: raise TypeError() or TypeError('Message')
        raise TypeError ('No Authorized')
```

In [ ]:
```
# Using the raising custom error
try:
    open_register('auto')
except TypeError as e:
    print(e)
```

In [ ]:
```
# Another example of raising custom error
try:
    a = int(input('Enter a value'))
    if a > 0:
        print(a)
    else:
        raise ValueError ('The number is negarive or zero')
```

```
except ValueError as e:
    print(e)
```

In [ ]:
```
x = "hello"
try:
    if not type(x) is int:
        raise TypeError("Only integers are allowed")
except TypeError as e:
    print(e)
```

In [ ]:
```

```

In [ ]:
```

```

---

### The Regular Expressions are optional materail

# `Regular Expressions`

- A **regular expression** describes a **search pattern** for **matching** characters in other strings
- Can help you extract data from unstructured text
- Can help you ensure that data is in the correct format before processing it

Regex functionality in Python resides in a module named re.

In [ ]:
```
s = 'test110boo'
```

In [ ]:
```
import re
```

In [ ]:
```
print(re.search('110' , s))
```

In [ ]:
```
if re.search('110', s):
    print('Found a match')
else:
    print('No match')
```

In [ ]:
```
('Found a match') if re.search('110', s) else ('No match')
```

### Matching Literal Characters

In [ ]:
```
'match' if re.fullmatch(s, 'test110boo') else 'NO match'
```

In [ ]:
```
'match' if re.fullmatch(s, 'test110goo') else 'NO match'
```

## Metacharacters, Character Classes and Quantifiers

- Regular expressions typically contain various special symbols called **metacharacters**:

| Regular expression metacharacters | -------- | `[]` `{}` `()` `\` `*` `+` `^` `$` `?` `.` `\|`

- `\` **metacharacter** begins each predefined **character class**
- Each matches a specific set of characters

In [ ]:
```
'Valid' if re.fullmatch(r'\d{5}', '02215') else 'Invalid'
```

In [ ]:
```
'Valid' if re.fullmatch(r'\d{5}', '9876') else 'Invalid'
```

- In `\d{5}`, `\d` is a character class representing a digit (0–9)
- A character class is a **regular expression escape sequence** that matches **one** character
- To match more than one, follow the character class with a **quantifier**
- `{5}` repeats `\d` five times to match five consecutive digits

## Other Predefined Character Classes

- To match any metacharacter as its **literal** value, precede it by a backslash ( `\` )
  - For example, `\\` matches a backslash ( `\` ) and `\$` matches a dollar sign ( `$` )

| Character class | Matches |
|---|---|
| `\d` | Any digit (0–9). |
| `\D` | Any character that is *not* a digit. |
| `\s` | Any whitespace character (such as spaces, tabs, and newlines). |
| `\S` | Any character that is *not* a whitespace character. |
| `\w` | Any **word character** (also called an **alphanumeric character**)—that is, any uppercase or lowercase letter, any digit, or an underscore. |
| `\W` | Any character that is *not* a word character. |

## Custom Character Classes

- Square brackets, `[]`, define a **custom character class** that matches a **single** character
- `[aeiou]` matches a lowercase vowel
- `[A-Z]` matches an uppercase letter
- `[a-z]` matches a lowercase letter
- `[a-zA-Z]` matches any lowercase or uppercase letter

```
In [ ]: 'Valid' if re.fullmatch('[A-Z][a-z]*', 'Wally') else 'Invalid'
```

```
In [ ]: 'Valid' if re.fullmatch('[A-Z][a-z]*', 'eva') else 'Invalid'
```

```
In [ ]: 'Match' if re.fullmatch('[^a-z]', 'A') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch('[^a-z]', 'a') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch('[*+$]', '*') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch('[*+$]', '!') else 'No match'
```

## `*` vs. `+` Quantifier

- `+` matches **at least one occurrence** of a subexpression
- `*` and `+` are **greedy**—they match as many characters as possible

```
In [ ]: 'Valid' if re.fullmatch('[A-Z][a-z]+', 'Wally') else 'Invalid'
```

```
In [ ]: 'Valid' if re.fullmatch('[A-Z][a-z]+', 'E') else 'Invalid'
```

## Other Quantifiers

- `?` **quantifier** matches **zero or one occurrences** of a subexpression

```
In [ ]: 'Match' if re.fullmatch('labell?ed', 'labelled') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch('labell?ed', 'labeled') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch('labell?ed', 'labellled') else 'No match'
```

- `labell?ed` matches `labelled` (the U.K. English spelling) and `labeled` (the U.S. English spelling), but not the misspelled word `labellled`
- `l?` indicates that there can be **zero or one more** `l` characters before the remaining literal `ed` characters

- Can match **at least *n* occurrences** of a subexpression with the **`{n,}` quantifier**

```
In [ ]: 'Match' if re.fullmatch(r'\d{3,}', '123') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch(r'\d{3,}', '1234567890') else 'No match'
```

```
In [ ]: 'Match' if re.fullmatch(r'\d{3,}', '12') else 'No match'
```

- Can match **between *n* and *m* (inclusive) occurrences** of a subexpression with the **`{n,m}` quantifier**

In [ ]:
```
'Match' if re.fullmatch(r'\d{3,6}', '123') else 'No match'
```

In [ ]:
```
'Match' if re.fullmatch(r'\d{3,6}', '123456') else 'No match'
```

In [ ]:
```
'Match' if re.fullmatch(r'\d{3,6}', '1234567') else 'No match'
```

In [ ]:
```
'Match' if re.fullmatch(r'\d{3,6}', '12') else 'No match'
```

## Function `search`—Finding the First Match Anywhere in a String

- **`search`** looks in a string for the *first* occurrence of a substring that matches a regular expression and returns a **match object** (of type **`SRE_Match`**) that contains the matching substring
- Match object's **`group`** method returns that substring

In [ ]:
```
result = re.search('Python', 'Python is fun')
```

In [ ]:
```
result.group() if result else 'not found'
```

- `search` returns `None` if the string does *not* contain the pattern

In [ ]:
```
result2 = re.search('fun!', 'Python is fun')
```

In [ ]:
```
result2.group() if result2 else 'not found'
```

- Can search for a match only at the *beginning* of a string with function **`match`**

## Ignoring Case with the Optional `flags` Keyword Argument

- Many `re` module functions receive an optional `flags` keyword argument
- Changes how regular expressions are matched

In [ ]:
```
result3 = re.search('Sam', 'SAM WHITE', flags=re.IGNORECASE)
```

In [ ]:
```
result3.group() if result3 else 'not found'
```

## Metacharacters That Restrict Matches to the Beginning or End of a String

- **`^`** **metacharacter** at the beginning of a regular expression (and not inside square brackets) is an anchor
- Indicaties that the expression matches only the *beginning* of a string

In [ ]:
```
result = re.search('^Python', 'Python is fun')
```

In [ ]:
```
result.group() if result else 'not found'
```

In [ ]:
```
result = re.search('^fun', 'Python is fun')
```

In [ ]:
```
result.group() if result else 'not found'
```

- **`$` metacharacter** at the end of a regular expression is an anchor indicating that the expression matches only the *end* of a string

In [ ]:
```
result = re.search('Python$', 'Python is fun')
```

In [ ]:
```
result.group() if result else 'not found'
```

In [ ]:
```
result = re.search('fun$', 'Python is fun')
```

In [ ]:
```
result.group() if result else 'not found'
```

## Function `findall` and `finditer`—Finding All Matches in a String

- **`findall`** finds *every* matching substring in a string
- Returns a list of the matching substrings

In [ ]:
```
contact = 'Wally White, Home: 555-555-1234, Work: 555-555-4321'
```

In [ ]:
```
re.findall(r'\d{3}-\d{3}-\d{4}', contact)
```

- **`finditer`** works like `findall`, but returns a lazy *iterable* of match objects

In [ ]:
```
for phone in re.finditer(r'\d{3}-\d{3}-\d{4}', contact):
    print(phone.group())
```

## Capturing Substrings in a Match

- Use **parentheses metacharacters**— **`(`** and **`)`** —to capture substrings in a match

In [ ]:
```
text = 'Charlie Cyan, e-mail: demo1@deitel.com'
```

In [ ]:
```
pattern = r'([A-Z][a-z]+ [A-Z][a-z]+), e-mail: (\w+@\w+\.\w{3})'
```

In [ ]:
```
result = re.search(pattern, text)
```

- The regular expression specifies two substrings to capture, each denoted by the metacharacters `(` and `)`
- `(` and `)` do *not* affect whether the `pattern` is found in the string `text`
- `match` function returns a match object *only* if the *entire* `pattern` is found in the string `text`
- `match` object's **`groups`** method returns a tuple of the captured substrings

In [ ]:
```
result.groups()
```

- `match` object's `group` method returns the *entire* match as a single string

In [ ]:
```
result.group()
```

- Access each captured substring by passing an integer to the `group` method

In [ ]:
```
result.group(1)
```

In [ ]:
```
result.group(2)
```